

Grafos implícitos

Recorrer sin construir el grafo

Carlos A Delgado S

Pontificia Universidad Javeriana Cali

Septiembre de 2026

Objetivos de la sesión

Al terminar la clase, usted podrá

- Justificar que los dos recorridos cuestan $\Theta(V + E)$ sobre lista de adyacencia, y decir qué cambia sobre las otras representaciones.
- Reconocer un grafo implícito: uno cuyos vértices y aristas se calculan con una regla en vez de guardarse.
- Recorrer una cuadrícula con DFS y con BFS sin construir la lista de adyacencia, y decir qué cuesta.
- Llevar un problema de juez a un grafo: decidir qué es un vértice, qué es una arista y qué recorrido contesta la pregunta.
- Reconocer cuándo el vértice no es la celda sino la celda más un estado.

Contenido

- 1 Recuento
- 2 Grafos implícitos
- 3 Un ejercicio tipo tarea: Rumor
- 4 Un ejercicio con estado: Robot
- 5 Errores comunes
- 6 Ejercicios
- 7 Referencias

Contenido

- 1 Recuento
- 2 Grafos implícitos
- 3 Un ejercicio tipo tarea: Rumor
- 4 Un ejercicio con estado: Robot
- 5 Errores comunes
- 6 Ejercicios
- 7 Referencias

Los dos recorridos

$DFS_{aux}(v)$

1. Marcar v como visitado
2. Para cada nodo u adyacente a v :
 - 2.1 Si u no ha sido visitado ejecutar $DFS_{aux}(u)$

Los dos recorridos

$BFS(G, s)$

1. Hacer $s.distancia = 0$ y agregar s a la cola P
2. Mientras la cola P no esté vacía:
 - 2.1 Retirar el frente w de la cola P
 - 2.2 Para cada nodo u adyacente a w :
 - 2.2.1 Si $u.distancia = \infty$: hacer $u.distancia = w.distancia + 1$ y agregar u a P

Lo que comparten

Los dos visitan exactamente los vértices alcanzables desde el arranque, cada uno una sola vez, y miran cada arista una vez por cada extremo. La profundidad marca al entrar; la amplitud, al encolar, y de regalo deja la distancia en aristas.

Lo que cuestan

Teorema

Recorrer un grafo $G = (V, E)$ en profundidad o en amplitud, guardado como lista de adyacencia, cuesta $\Theta(V + E)$.

Demostración

Se procede contando por separado lo que aporta cada vértice y lo que aporta cada arista. Cada vértice se marca una sola vez, porque antes de entrar se pregunta por la marca: $\Theta(V)$ en total. Estando en u se recorre su lista de vecinos, de $\delta(u)$ entradas, y como u se visita una vez esa lista se recorre una vez; la suma sobre todos los vértices es $\sum_u \delta(u)$, que vale $2|E|$ en el grafo no dirigido y $|E|$ en el dirigido: $\Theta(E)$. Sumando, $\Theta(V + E)$.

Lo que cuestan

Conclusión

Por lo tanto, se puede concluir que ambos recorridos cuestan $\Theta(V + E)$ sobre lista de adyacencia. Sobre matriz de adyacencia cuestan $\Theta(V^2)$, porque los vecinos de u obligan a recorrer su fila completa; sobre lista de aristas, $\Theta(V \cdot E)$.

Contenido

1 Recuento

2 Grafos implícitos

3 Un ejercicio tipo tarea: Rumor

4 Un ejercicio con estado: Robot

5 Errores comunes

6 Ejercicios

7 Referencias

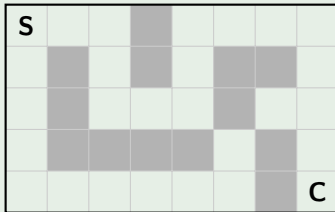
Un simio y una taza de café

El problema

Un laberinto de R filas y C columnas. Cada celda es pared o es libre. El simio está en una celda libre y el café en otra. En cada paso el simio se mueve a una celda libre contigua: arriba, abajo, izquierda o derecha. ¿Puede llegar al café? Y si puede, ¿en cuántos pasos como mínimo?

Un simio y una taza de café

El laberinto de hoy



$R = 5$, $C = 8$. S es el simio, C el café y las celdas grises son pared.

Un simio y una taza de café

¿Entendimos el problema?

Entrada: el laberinto y las dos posiciones. **Salida:** si hay camino, y el número mínimo de pasos. Con el dedo: el corredor de abajo parece la ruta obvia y termina en un callejón; la que llega baja dos filas, cruza por debajo de la pared, vuelve a subir y sigue por arriba hasta la última columna. Contar esos pasos a mano ya es un recorrido.

La cuadrícula es un grafo

Definición (Grafo implícito)

Un grafo cuyos vértices y aristas no están guardados en ninguna estructura: hay una regla que dice qué es un vértice y una regla que, dado un vértice, produce sus vecinos. El recorrido llama a esa regla cada vez que necesita la lista de adyacencia de alguien.

En el laberinto

- **Vértices:** las celdas libres (r, c) .
- **Aristas:** $\{(r, c), (r', c')\}$ cuando las dos celdas son libres y contiguas: $|r - r'| + |c - c'| = 1$.
- **Vecinos de (r, c) :** a lo sumo cuatro, y se obtienen sumando $(1, 0)$, $(-1, 0)$, $(0, 1)$ y $(0, -1)$ y descartando lo que se sale del tablero o cae en pared.

La cuadrícula es un grafo

Por qué no se construye

La lista de adyacencia de un laberinto de 500×500 tendría 250 000 listas y hasta un millón de entradas, todas deducibles de una suma. Se gastaría $\Theta(R \cdot C)$ en memoria y en tiempo solo para escribir lo que la regla ya dice.

La regla de los vecinos

```
DR = [1, -1, 0, 0]
```

```
DC = [0, 0, 1, -1]
```

```
def es_libre(laberinto, r, c):  
    # (r, c) existe y no es pared  
    R = len(laberinto)  
    C = len(laberinto[0])  
    return r >= 0 and r < R and c >= 0 and c < C and laberinto[r][c] != "#"
```

```
def vecinos(laberinto, r, c):  
    # Las celdas contiguas a (r, c) por las que se puede pasar  
    res = []  
    k = 0  
    while k < 4:  
        nr = r + DR[k]  
        nc = c + DC[k]  
        if es_libre(laberinto, nr, nc):  
            res.append((nr, nc))  
        k = k + 1  
    return res
```

La regla de los vecinos

Los dos arreglos de desplazamientos

$DR[k]$ y $DC[k]$ son el k -ésimo movimiento. Escribirlos así evita cuatro `if` y sirve igual para ocho direcciones o para los saltos de un caballo: cambia la tabla, no el recorrido.

¿Puede llegar? Profundidad

```
def dfs(laberinto, r, c, visitado):  
    # Marca (r, c) y sigue por cada vecino que falte  
    visitado[r][c] = True  
    for celda in vecinos(laberinto, r, c):  
        if not visitado[celda[0]][celda[1]]:  
            dfs(laberinto, celda[0], celda[1], visitado)  
  
def puede_llegar(laberinto, r0, c0, rf, cf):  
    # True si hay camino de (r0, c0) a (rf, cf)  
    R = len(laberinto)  
    C = len(laberinto[0])  
    visitado = []  
    r = 0  
    while r < R:  
        visitado.append([False] * C)  
        r = r + 1  
    dfs(laberinto, r0, c0, visitado)  
    return visitado[rf][cf]
```

¿Puede llegar? Profundidad

Lo único que cambió

Es el *DFS**Aux* de siempre. Donde antes decía $G[u]$ ahora dice `vecinos(laberinto, r, c)`, y la marca es una matriz porque el vértice tiene dos coordenadas.

¿En cuántos pasos? Amplitud

```
from collections import deque

def distancias(laberinto, r0, c0):
    # d[r][c] = pasos minimos desde (r0, c0); -1 si no se llega
    R = len(laberinto)
    C = len(laberinto[0])
    d = []
    r = 0
    while r < R:
        d.append([-1] * C)
        r = r + 1
    d[r0][c0] = 0
    cola = deque()
    cola.append((r0, c0))
    while len(cola) > 0:
        actual = cola.popleft()
        r = actual[0]
        c = actual[1]
        for celda in vecinos(laberinto, r, c):
            if d[celda[0]][celda[1]] == -1:
                d[celda[0]][celda[1]] = d[r][c] + 1
                cola.append(celda)
    return d
```

¿En cuántos pasos? Amplitud

La distancia hace de marca

Igual que sobre la lista de adyacencia: -1 es “no visto”, y la distancia se fija al encolar. La respuesta a las dos preguntas está en $d[rf][cf]$: si es -1 no se llega, y si no, ese es el mínimo de pasos.

Las capas sobre el laberinto

```
distancias(LABERINTO, 0, 0)
```

0	1	2			8	9	10	11
1		3		7				12
2		4	5	6		14	13	
3					10		14	
4	5	6	7	8	9		15	

Cada celda lleva su distancia al simio. El café queda a 15 pasos por el camino verde. El corredor de abajo llega hasta la celda marcada 10 y ahí se cierra: la amplitud lo explora igual, y lo descarta sola.

Las capas sobre el laberinto

El mismo Teorema 22.5

La cuadrícula es un grafo no dirigido donde todas las aristas valen lo mismo, así que la primera vez que la amplitud toca una celda es por un camino más corto. Nada del argumento dependía de que el grafo estuviera guardado.

Lo que cuesta

$$\Theta(R \cdot C)$$

El grafo tiene a lo sumo $R \cdot C$ vértices y cada uno tiene a lo sumo cuatro aristas, así que $E \leq 2 \cdot R \cdot C$. El $\Theta(V + E)$ de los recorridos se convierte en $\Theta(R \cdot C)$, y calcular los vecinos cuesta $\Theta(1)$ por celda: la regla no agrega nada al orden.

La recursión sobre una cuadrícula grande

Un laberinto de 500×500 con un corredor largo lleva la profundidad recursiva a 250 000 llamadas anidadas, y Python se detiene mucho antes de eso. Sobre cuadrículas, la amplitud con cola o la profundidad con pila explícita son las versiones que corren.

Contenido

- 1 Recuento
- 2 Grafos implícitos
- 3 Un ejercicio tipo tarea: Rumor**
- 4 Un ejercicio con estado: Robot
- 5 Errores comunes
- 6 Ejercicios
- 7 Referencias

Rumor

Codeforces 893C — Rumor

<https://codeforces.com/problemset/problem/893/C>

El problema

En una ciudad hay n personajes; algunos pares son amigos. Sobornar al personaje i cuesta c_i monedas y hace que empiece a contar un rumor; quien lo oye se lo cuenta a todos sus amigos, gratis, y así sigue. ¿Cuál es el mínimo de monedas para que todos los n conozcan el rumor?

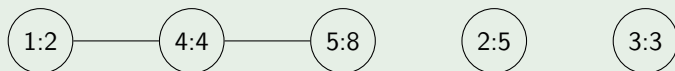
Entrada y salida

Entrada: varios casos. Cada uno trae n y m ($1 \leq n \leq 2 \cdot 10^4$, $0 \leq m \leq 4 \cdot 10^4$), la línea con $c_1, \dots, c_n$ y m pares de amigos, numerados de 1 a n . Termina con $n = m = 0$. **Salida:** una línea por caso con el mínimo de monedas.

¿Entendimos el problema?

El primer caso de la muestra

$n = 5$, $m = 2$, costos 2, 5, 3, 4, 8, amigos (1, 4) y (4, 5). Respuesta: 10.



Tres grupos que no se hablan entre sí: $\{1, 4, 5\}$, $\{2\}$ y $\{3\}$. En cada grupo basta sobornar a uno, y conviene el más barato: $2 + 5 + 3 = 10$.

¿Por qué 10 y no 2?

Sobornar solo al 1 deja el rumor encerrado en su grupo: el 2 y el 3 no tienen amigos y nunca lo oyen. Cada grupo necesita su propio soborno.

El grafo y el algoritmo

Qué es qué

- **Vértices:** los personajes. **Aristas:** las amistades, no dirigidas.
- “Quien lo oye se lo cuenta a todos sus amigos” es exactamente *el rumor llega a todo lo alcanzable*. Un soborno cubre un componente conexo completo, y nada más.
- La pregunta es entonces: **por cada componente, su menor costo,** y la suma de esos mínimos.

El grafo y el algoritmo

OroMinimo(G, c)

1. Marcar todos los nodos como no visitados; hacer $total = 0$
2. Para cada nodo u de G :
 - 2.1 Si u no ha sido visitado:
 - 2.1.1 Recorrer el componente de u llevando el menor c
 - 2.1.2 Hacer $total = total +$ ese menor
3. Devolver $total$

Es el ciclo externo de siempre

El paso 2 es el $DFS(G)$ completo: cada vez que arranca un recorrido nuevo es porque encontró un componente nuevo. Lo único que se agrega es un acumulador dentro del recorrido.

El código

```
def minimo_del_componente(G, s, visitado, costo):  
    # Recorre el componente de s con una pila y devuelve su menor  
    ↪ costo  
    menor = costo[s]  
    visitado[s] = True  
    pila = [s]  
    while len(pila) > 0:  
        u = pila.pop()  
        if costo[u] < menor:  
            menor = costo[u]  
        for v in G[u]:  
            if not visitado[v]:  
                visitado[v] = True  
                pila.append(v)  
    return menor
```

El código

```
def oro_minimo(n, costo, G):  
    # Suma el minimo de cada componente; el ciclo externo los descubre  
    visitado = [False] * n  
    total = 0  
    u = 0  
    while u < n:  
        if not visitado[u]:  
            total = total + minimo_del_componente(G, u, visitado,  
                                                    ↪ costo)  
            u = u + 1  
    return total
```

Costo

Un recorrido completo sobre lista de adyacencia: $\Theta(n + m)$. Con $n = 2 \cdot 10^4$ y $m = 4 \cdot 10^4$ son unas 10^5 operaciones por caso.

El código

Aquí la marca va al apilar

Con n hasta $2 \cdot 10^4$ la recursión de *DFS**Aux* puede pasarse del límite de Python. La pila explícita marca al apilar para que nadie entre dos veces; el orden de visita cambia, el componente que se recorre no.

Contenido

- 1 Recuento
- 2 Grafos implícitos
- 3 Un ejercicio tipo tarea: Rumor
- 4 Un ejercicio con estado: Robot**
- 5 Errores comunes
- 6 Ejercicios
- 7 Referencias

Robot

El problema

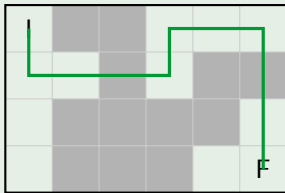
Un robot patrulla una cuadrícula de $m \times n$. Se mueve a una celda contigua por paso. Algunas celdas tienen obstáculos: entrar en una obliga a usar el turbo, y el robot **no puede pasar por más de k celdas con obstáculo seguidas**. Hay que hallar el mínimo de pasos de $(1, 1)$ a (m, n) , que son celdas libres, o -1 si no se puede.

Entrada y salida

Entrada: el número de casos; por caso, m y n ($1 \leq m, n \leq 100$), luego k ($0 \leq k \leq 100$), y m filas de n enteros: 1 es obstáculo, 0 es libre. **Salida:** una línea por caso con el número de pasos, o -1 .

¿Entendimos el problema?

Un caso de la muestra: $m = 4$, $n = 6$, $k = 1$. Respuesta: 10



El camino verde tiene diez pasos y pasa por dos obstáculos: el de la fila 2 columna 3 y el de la fila 2 columna 6. Nunca dos seguidos, así que con $k = 1$ es válido.

¿Entendimos el problema?

Con $k = 0$

El mismo tablero sin turbo no tiene solución: para entrar a la esquina final hay que atravesar la fila de obstáculos que la rodea.

El primer intento, y por qué falla

BFS sobre las celdas, contando la racha

La idea que sale primero: recorrer la cuadrícula como el laberinto del simio, y llevar en cada celda cuántos obstáculos seguidos se llevan al llegar a ella. Si al entrar a un obstáculo la racha pasa de k , no se entra.

La misma celda con dos rachas distintas

A una celda libre se puede llegar con racha 0; a una celda con obstáculo, con racha 1, 2, ... hasta k , según por dónde se vino. Si se marca la celda como visitada la primera vez, se descarta la segunda llegada aunque traiga una racha menor que sí deja seguir. La marca sobre la celda está tirando caminos válidos.

El primer intento, y por qué falla

Lo que dice el error

El vértice del grafo **no es la celda**. Dos llegadas a la misma celda con rachas distintas son dos situaciones distintas del robot, y el grafo tiene que distinguirlas.

El grafo implícito de los estados

Qué es qué

- **Vértices:** los estados (r, c, t) , con t el número de celdas con obstáculo que el robot lleva seguidas al estar en (r, c) ; $0 \leq t \leq k$.
- **Aristas:** de (r, c, t) a (r', c', t') cuando las celdas son contiguas y $t' = t + 1 \leq k$ si (r', c') es obstáculo, o $t' = 0$ si es libre.
- **Arranque:** $(0, 0, 0)$. **Llegada:** $(m - 1, n - 1, 0)$, porque la esquina final es libre.

Y otra vez es amplitud

Todas las aristas valen un paso, así que la primera vez que la amplitud toca $(m - 1, n - 1, 0)$ es por el camino más corto. La regla de los vecinos es la de la cuadrícula más la cuenta de la racha; el recorrido no cambia.

El grafo implícito de los estados

Costo

Hay $m \cdot n \cdot (k + 1)$ estados y cada uno tiene a lo sumo cuatro aristas:
 $\Theta(m \cdot n \cdot k)$. Con $100 \cdot 100 \cdot 101$ son un millón de estados por caso.

El código

```
def tabla_vacia(m, n, k):  
    # d[r][c][t] = -1 para todos los estados: ninguno visto todavia  
    d = []  
    r = 0  
    while r < m:  
        fila = []  
        c = 0  
        while c < n:  
            fila.append([-1] * (k + 1))  
            c = c + 1  
        d.append(fila)  
        r = r + 1  
    return d
```

El código

```
def vecinos_robot(m, n, k, a, r, c, t):  
    # Los estados a los que se pasa desde (r, c, t) en un movimiento  
    res = []  
    i = 0  
    while i < 4:  
        nr = r + DR[i]  
        nc = c + DC[i]  
        if nr >= 0 and nr < m and nc >= 0 and nc < n:  
            nt = 0  
            if a[nr][nc] == 1:  
                nt = t + 1  
            if nt <= k:  
                res.append((nr, nc, nt))  
        i = i + 1  
    return res
```

El código

```
def movimientos_minimos(m, n, k, a):  
    # BFS sobre los estados (r, c, t); -1 si (m-1, n-1) no se alcanza  
    d = tabla_vacia(m, n, k)  
    d[0][0][0] = 0  
    cola = deque()  
    cola.append((0, 0, 0))  
    while len(cola) > 0:  
        e = cola.popleft()  
        for s in vecinos_robot(m, n, k, a, e[0], e[1], e[2]):  
            if d[s[0]][s[1]][s[2]] == -1:  
                d[s[0]][s[1]][s[2]] = d[e[0]][e[1]][e[2]] + 1  
                cola.append(s)  
    return d[m - 1][n - 1][0]
```

El código

La marca tiene tres índices

$d[r][c][t]$ es la distancia *al estado*, y es lo que permite que la misma celda entre a la cola con dos rachas distintas. Comparado con el laberinto del simio, lo único que cambió es la regla de los vecinos: `vecinos_robot` lleva la racha y descarta lo que pasa de k . El recorrido es el mismo.

Contenido

- 1 Recuento
- 2 Grafos implícitos
- 3 Un ejercicio tipo tarea: Rumor
- 4 Un ejercicio con estado: Robot
- 5 Errores comunes**
- 6 Ejercicios
- 7 Referencias

Los que más cuestan sobre cuadrículas

Al calcular los vecinos

- Salirse del tablero: la regla revisa $0 \leq r < R$ y $0 \leq c < C$ antes de mirar la celda. En Python un índice negativo no falla, lee desde el final, y el error queda escondido.
- Confundir fila con columna al leer la entrada, o la numeración desde 1 del enunciado con la desde 0 del programa.

Los que más cuestan sobre cuadrículas

Al recorrer

- Marcar la celda cuando el vértice es la celda más un estado.
- Usar profundidad recursiva sobre una cuadrícula grande.
- Buscar la distancia mínima con profundidad: la profundidad llega, pero no por el camino corto.
- Olvidar el ciclo externo cuando la pregunta es por todos los componentes, como en Rumor.

Contenido

- 1 Recuento
- 2 Grafos implícitos
- 3 Un ejercicio tipo tarea: Rumor
- 4 Un ejercicio con estado: Robot
- 5 Errores comunes
- 6 Ejercicios**
- 7 Referencias

Para enviar al juez

UVa 439 — Knight Moves

Enunciado: <https://onlinejudge.org/external/4/439.pdf>

Envío: https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=380

Un grafo implícito que no es una cuadrícula de vecinos contiguos: los vértices son las 64 casillas y las aristas, los ocho saltos del caballo. La tabla DR, DC tiene ocho entradas y el resto es la amplitud de hoy.

Para enviar al juez

UVa 469 — Wetlands of Florida

Enunciado: <https://onlinejudge.org/external/4/469.pdf>

Envío: https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=410

Dan un mapa de tierra y agua y preguntan, para varias celdas, el tamaño del lago que las contiene. Es el componente conexo de la celda; cuente mientras recorre. La trampa está en la lectura de la entrada.

Para enviar al juez

UVa 10653 — Bombs! NO they are Mines!!

Enunciado: <https://onlinejudge.org/external/106/10653.pdf>

Envío: https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=1594

El laberinto del simio tal cual: celdas bloqueadas y la distancia mínima entre dos posiciones. La entrada describe las minas por filas, con cuántas hay en cada una, y hay que armar la cuadrícula antes de recorrer. Con 1000×1000 celdas, la amplitud cabe y la profundidad recursiva no.

Para practicar en papel

Sobre el laberinto de hoy

- 1 Corra la profundidad desde el simio siguiendo el orden de DR, DC y escriba el orden en que se visitan las primeras diez celdas. Compare con el orden de la amplitud.
- 2 Si el simio también pudiera moverse en diagonal, ¿cuántas entradas tendrían DR y DC y en cuánto quedaría la distancia al café?
- 3 Dibuje un laberinto de 4×4 donde la profundidad llegue al café por un camino de más del doble de la distancia mínima.

Para practicar en papel

Sobre los estados

- 1 En Robot, ¿qué pasa si la esquina final tuviera obstáculo? ¿En qué estado habría que buscar la respuesta?
- 2 Un robot que puede usar el turbo a lo sumo k veces en *todo* el recorrido, no k seguidas. ¿Cuál es el estado ahora, y cuántos hay?
- 3 Escriba *DFS*_{Aux} y *BFS* sobre matriz de adyacencia y sobre lista de aristas y diga cuánto cuesta cada recorrido en términos de V y E .

Lo que sigue

Lo que los recorridos todavía no dicen

La profundidad deja un árbol y clasifica las aristas que no usó: unas apuntan a un ancestro, otras a un descendiente, otras cruzan. Con esa clasificación se detectan ciclos, se ordena un grafo dirigido y se encuentran los puntos por donde un grafo se parte.

Para investigar

Busque qué es una **arista de retroceso** en un recorrido en profundidad, y qué tiene que ver con que un grafo dirigido tenga ciclos.

Contenido

- 1 Recuento
- 2 Grafos implícitos
- 3 Un ejercicio tipo tarea: Rumor
- 4 Un ejercicio con estado: Robot
- 5 Errores comunes
- 6 Ejercicios
- 7 Referencias**

Referencias

-  Cormen, T. H., Leiserson, C. E., Rivest, R. L., Stein, C. *Introduction to Algorithms*, 3rd ed. MIT Press, 2009. Sección 22.2 (pp. 594–602) y Sección 22.3 (pp. 603–612).
-  Halim, S., Halim, F., Effendy, S. *Competitive Programming 4*. Lulu, 2020. Sección 4.2, recorridos, y su tratamiento de las cuadrículas como grafos implícitos.
-  Codeforces Round 449, problema 893C, *Rumor*.
<https://codeforces.com/problemset/problem/893/C>